Coding and Solution

Phase 5: Project Development Phase

1. Implementation Overview

The implementation connects a React frontend to a FastAPI backend. The user supplies preferences in the wizard; the backend validates them, obtains a structured itinerary from the AI provider (or the fallback generator), computes the budget and returns the trip. The frontend displays the plan and offers editing, saving and printing.

2. Request Validation

The GenerateItineraryRequest schema requires a destination of at least two characters and date strings in YYYY-MM-DD format. It limits travellers to 1 through 50, requires a budget of at least 50, restricts traveller type to solo, couple, family, friends or business, style to budget, balanced or luxury, and pace to relaxed, moderate or packed. A model validator rejects an end date earlier than the start date.

3. Prompt and Provider Call

The prompt describes the trip parameters and asks for strictly valid JSON with no markdown. For Gemini the request uses generateContent with a JSON response type and a low temperature; for OpenAI it uses chat completions with a JSON response format. Both calls use a configurable timeout (45 seconds by default).

4. Hydration and Fallback

The reply is parsed and every day and activity receives an identifier, default time slot, duration, cost, category and booking tip when missing. If the provider is not configured, or any step fails, the heuristic generator builds a plan so that the user still receives a result. The frontend adds a second fallback by generating a plan locally if the backend cannot be reached.

5. Budget Logic

BudgetService adds the accommodation cost of each day and the cost of each activity. Food and meal activities count as food, transit and transport as transit, and all other activities as activities. It returns the target, total estimate, category totals, variance, an over-budget flag and warning messages.

6. Trip Operations and Ownership

The trips router lists trips, creates a trip with new identifiers for its days and activities, retrieves one trip, updates its metadata, deletes it, regenerates a day and exports printable HTML. For signed-in users it compares the trip owner with the authenticated user and returns 403 when they differ. When Supabase is configured, the list route reads from the database and the create route inserts the trip row.

7. Authentication

The get_current_user dependency accepts a bearer token and verifies it through Supabase. If demo mode is enabled it also accepts demo tokens and requests without a token. In the frontend, AuthContext uses Supabase when the environment variables are present and otherwise creates a demo user in localStorage.

8. Frontend Rendering and Print

The itinerary page shows a header with the Print / PDF button, a budget summary and day tabs. Elements marked no-print are hidden by the print stylesheet and all days are rendered when printing. The print helper temporarily sets the document title so that a saved PDF has a meaningful file name.

9. Testing the Implementation

The backend has 17 pytest tests and the frontend has 6 Vitest tests, covering health, validation, budget, AI output structure, trip operations, export, formatters and sample data. The tests are described in the testing document. Source code and test files alone do not establish that every path has been exercised in the deployed environment.